Assignment 3: Simulating Dynamical Systems

1 Introduction

In this assignment the student will learn how to simulate dynamical systems using integration schemes for ordinary differential equations. This will be done by extending simutils.py with useful functions to simulate dynamical systems

After completing this assignment the student will learn:

- How to implement numerical integration schemes for simulating dynamical systems described by ordinary differential equations

2 Preliminaries

Before describing the steps needed to complete todays assignment, some fundamentals are needed. Specifically, the fundamentals of ordinary differential equations. This section is is based on Lecture 4.

2.1 Simulating dynamical systems

By simulating a system we typically mean numerically solving the initial value problem

$$\dot{\boldsymbol{x}}(t) = f\left(t, \boldsymbol{x}(t)\right), \qquad \boldsymbol{x}(0) = \boldsymbol{x}_0$$

recall that the solution to an ODE is function of time, i.e. $\boldsymbol{x}(t)$. For instance for the initial value problem

$$3\ddot{y}(t) + 3\dot{y}(t) + 10y(t) = 4\delta(t), \dot{y}(0) = 0, y(0) = 0$$

is

$$y(t) = 0.76e^{-0.5t} \sin 1.76t. \tag{1}$$

With the solution $y(t)$ we can compute y for any value of t for that particular initial condition and input. In general it not always so easy or feasible to solve ODEs, especially if the are nonlinear. Instead we can use numerical techniques to approximate a solution.

2.1.1 Taylor Series

We know that a function f(x) can be represented by a taylor series at a point a

$$\begin{aligned} f(x) = f(a) + \left.\frac{df(x)}{dx}\right|_{x=a}(x-a) + \frac{1}{2!}\left.\frac{d^2f(x)}{dx^2}\right|_{x=a}(x-a)^2 \\ + \frac{1}{3!}\left.\frac{d^3f(x)}{dx^3}\right|_{x=a}(x-a)^3 + \cdots = \sum_{n=0}^{\infty}\frac{1}{n!}\left.\frac{d^nf(x)}{dx^n}\right|_{x=a}(x-a)^n \end{aligned}$$

By truncating the series to p terms, we can obtain an approximation of $f(x)$ near a

$$f(x) \approx \sum_{n=0}^{p}\frac{1}{n!}\left.\frac{d^nf(x)}{dx^n}\right|_{x=a}(x-a)^n$$

which we call a pth order Taylor polynomial. If we move the value of x further away from a, then the approximation will get worse until it is unusable. Therefore if we know the value of f at some a then we can use a Taylor polynomial to find approximate values for f near a.

In an initial value problem, the initial value of our unknown function is given by the initial condition. Therefore since we know the value of the solution at the initial time, we can use a Taylor polynomial to find an approximate value at some t close to the initial time.

Given an initial value problem $\dot{\boldsymbol{x}} = f(t, \boldsymbol{x})$ with initial condition $\boldsymbol{x}(0) = \boldsymbol{x}_0$. Then the value $\boldsymbol{x}(t+h)$ at can be approximated as

$$\boldsymbol{x}(h) = \boldsymbol{x}(0) + \frac{d\boldsymbol{x}(0)}{dt}h + \frac{1}{2!}\frac{d^2\boldsymbol{x}(0)}{dt^2}h^2 + \frac{1}{3!}\frac{d^3\boldsymbol{x}(0)}{dt^3}h^3 + \cdots \frac{1}{p!}\frac{d^p\boldsymbol{x}(0)}{dt^p}h^p$$

Now that we have an approximate of $\boldsymbol{x}(t)$ at h, we can use it to approximate the value $\boldsymbol{x}(t)$ at $2h$

$$\boldsymbol{x}(2h) = \boldsymbol{x}(h) + \frac{d\boldsymbol{x}(h)}{dt}h + \frac{1}{2!}\frac{d^2\boldsymbol{x}(h)}{dt^2}h^2 + \frac{1}{3!}\frac{d^3\boldsymbol{x}(h)}{dt^3}h^3 + \cdots \frac{1}{p!}\frac{d^p\boldsymbol{x}(h)}{dt^p}h^p$$

By repeatedly doing this in an recursive manner we are obtaining a numerical approximation of the solution to the given initial value problem. We can more generally state this procedure as. The approximation of $\boldsymbol{x}(t+h)$ is obtained from the approximation at $\boldsymbol{x}(t)$ by

$$\boldsymbol{x}(t+h) = \sum_{n=0}^{p} \frac{1}{n!}\frac{d^n x(t)}{dt^n}h^n$$

Clearly if $\boldsymbol{x}(t)$ is a bad approximation then so will $\boldsymbol{x}(t+h)$ be and as you iterate with computing each new approximation based on a bad approximationit is not surprising that $\boldsymbol{x}(t)$ will start to diverge from the true solution. What we are essentially doing is extrapolation.

2.1.2 Forward Euler

We previously introduced the Forward Euler's method, which is based on the Taylor series approach from the previous section. Consider a first order differential equation

$$\dot{\boldsymbol{x}}(t) = f\left(t, \boldsymbol{x}(t)\right)$$

with initial value $\boldsymbol{x}(0) = \boldsymbol{x}_0$. Use a first order Taylor polynomial to obtain a approximate $\boldsymbol{x}(t+h)$

$$\boldsymbol{x}(t+h) = \boldsymbol{x}(t) + \frac{dx(t)}{dt}h = \boldsymbol{x}(t) + h \cdot f\left(t, \boldsymbol{x}(t)\right)$$

Then if we divide time into equally spaced points t_k with $h = t_k - t_{k-1}$ then we get the following reccurrence relation

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + h \cdot f(t_k, \boldsymbol{x}_k), \qquad k = 0, 1, ...$$

This will typically not yield a good approximate, but it can be improved by reducing h at the cost of more computational effort. However, at some point reducing h will increase other numerical errors such as floating point truncation. The reason for the poor approximation is that Forward Euler's method uses the tangent $(f(t_k, \boldsymbol{x}_k))$ at the current time to approximate where $\boldsymbol{x}(t)$ is at $t+h$. Another way to think of it is that Forward Euler uses $f(t, \boldsymbol{x}_k)$ as the direction of where to go from t to $t+h$. This only works well if $f(t_k, \boldsymbol{x}_k)$ stays approximately constant from t_k to t_{k+1}. To improve the situation there are generally two strategies

- Use a higher order Taylor polynomial.
- Choose a better direction.

2.2 Leapfrog integration

By using a second order Taylor polynomial we get

$$\boldsymbol{x}(t+h) = \boldsymbol{x}(t) + \frac{d\boldsymbol{x}(t)}{dt}h + \frac{1}{2}\frac{d^2\boldsymbol{x}(t)}{dt^2}h^2$$

In many applications the solution we are approximating represents the position of some thing, like a particle, a vehicle or a celestial body. Therefore we will assume that $\boldsymbol{x}(t)$ represents a position. Then the polynomial is equivalent to

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{v}_k h + \frac{1}{2}\boldsymbol{a}_k h^2$$

which is just the kinematics formula. This assumes that the acceleration remains constant over h, which is a good approximation for near circular orbits. The method requires that we know the position as well as the velocity at each t_k and therefore takes the form

$$\begin{aligned}\boldsymbol{x}_{k+1} &= \boldsymbol{x}_k + \boldsymbol{v}_k h + \frac{1}{2}\boldsymbol{a}_k h^2 \\ \boldsymbol{v}_{k+1} &= \boldsymbol{v}_k + \boldsymbol{a}_k h\end{aligned}$$

Instead of using the second equation to update $\boldsymbol{v}$ for the next time step, we could first compute $\boldsymbol{v}_{k+1}$ and then use the average velocity in the first equation

$$\boldsymbol{v} = \frac{\boldsymbol{v}_{k+1} + \boldsymbol{v}_k}{2} = \boldsymbol{v}_k + \frac{1}{2}\boldsymbol{a}_k h$$

where we have inserted for $\boldsymbol{v}_{k+1}$. Noting that $\boldsymbol{v}h = \boldsymbol{v}_k h + \frac{1}{2}\boldsymbol{a}_k h^2$ we can write the kinematic formula as

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{v} \cdot h$$

Knowing $\boldsymbol{v}_k$ is still required to compute $\boldsymbol{v}$, but if we solve the equation for the average velocity with respect to v_{k+1} we get $\boldsymbol{v}_{k+1} = 2\boldsymbol{v} - \boldsymbol{v}_k$. Together this gives the following integration scheme

$$\begin{aligned}\boldsymbol{v} &= \boldsymbol{v}_k + \frac{1}{2}\boldsymbol{a}_k h \\ \boldsymbol{x}_{k+1} &= \boldsymbol{x}_k + \boldsymbol{v}h \\ \boldsymbol{v}_{k+1} &= 2\boldsymbol{v} - \boldsymbol{v}_k\end{aligned}$$

where is $\boldsymbol{a}_k$ computed from the dynamics

$$\boldsymbol{a}_k = \frac{1}{m}\sum \boldsymbol{f}(t_k, \boldsymbol{x}_k)$$

2.3 Verlet integration

When truncating the Taylor series to a second order Taylor polynomial

$$\begin{aligned}\boldsymbol{x}(t+h) &= \boldsymbol{x}(t) + \frac{d\boldsymbol{x}(t)}{dt}h + \frac{1}{2}\frac{d^2\boldsymbol{x}(t)}{dt}h^2 + \frac{1}{6}\frac{d^3\boldsymbol{x}(t)}{dt}h^3, ... \\ \boldsymbol{x}(t+h) &\approx \boldsymbol{x}(t) + \frac{d\boldsymbol{x}(t)}{dt}h + \frac{1}{2}\frac{d^2\boldsymbol{x}(t)}{dt^2}h^2\end{aligned}$$

where we are assuming that all terms of h^3 and higher are negligable. We can arrive at a better approximation if we write the Taylor series for both $\boldsymbol{x}(t+h)$ and $\boldsymbol{x}(t-h)$

$$\boldsymbol{x}(t+h) = \boldsymbol{x}(t) + \frac{d\boldsymbol{x}(t)}{dt}h + \frac{1}{2}\frac{d^2\boldsymbol{x}(t)}{dt^2}h^2 + \frac{1}{6}\frac{d^3\boldsymbol{x}(t)}{dt^3}h^3, ...$$
$$\boldsymbol{x}(t-h) = \boldsymbol{x}(t) - \frac{d\boldsymbol{x}(t)}{dt}h + \frac{1}{2}\frac{d^2\boldsymbol{x}(t)}{dt^2}h^2 - \frac{1}{6}\frac{d^3\boldsymbol{x}(t)}{dt^3}h^3, ...$$

Adding them together will cause all the terms with odd powers of h to cancel out giving us

$$\boldsymbol{x}(t+h) + \boldsymbol{x}(t-h) = 2\boldsymbol{x}(t) + \frac{d^2\boldsymbol{x}(t)}{dt}h^2 + \frac{1}{6}\frac{d^4\boldsymbol{x}(t)}{dt}h^4, ...$$

Taking the second order polynomial we end up only truncating terms h^4 and above, therefore we have essentially gained an order of accuracy. Assuming again that $\boldsymbol{x}(t)$ is position we get

$$\boldsymbol{x}_{k+1} = 2\boldsymbol{x}_k - \boldsymbol{x}_{k-1} + \boldsymbol{a}_k h^2$$

instead of the velocity we now have $\boldsymbol{x}_{k-1}$ instead. In fact with a bit of maniupulation we can arrive at

$$\begin{aligned}\boldsymbol{x}_{k+1} &= \boldsymbol{x}_k + \boldsymbol{x}_k - \boldsymbol{x}_{k-1} + \boldsymbol{a}_k h^2 \\ &= \boldsymbol{x}_k + \left(\frac{\boldsymbol{x}_k - \boldsymbol{x}_{k-1}}{h} + \boldsymbol{a}_k h\right) h \\ &= \boldsymbol{x}_k + \boldsymbol{v}h\end{aligned}$$

This is the same form as the previous method, $\boldsymbol{v}$ has been replaced by

$$\boldsymbol{v} = \frac{\boldsymbol{x}_k - \boldsymbol{x}_{k-1}}{h} + \boldsymbol{a}_k h$$

Where instead of integrating $\boldsymbol{v}_k$ like in the leapfrog method which will accumulate error over time, we replace it with the average velocity from t_{k-1} to t_k. There are two things not note when implementing verlet, first of all computing $\boldsymbol{x}(t)$ at t_1 requires $\boldsymbol{x}(t)$ at t_{-1} which we obviously don't have. Instead we use the kinematics equation for the first time step, then switch to verlet integration. The second thing to note is that sometimes we still require the knowledge of the velocity vector. Therefore we can use the average as an approximation. Then verlet integration algorithm is

Given $\boldsymbol{x}_0, \boldsymbol{v}_0$

Step 1: $\boldsymbol{x}_1 = \boldsymbol{x}_0 + \boldsymbol{v}_0 h + \frac{1}{2}\boldsymbol{a}_0 h^2$

Step 2: for k = 1,.... Compute $\boldsymbol{a}_k$ as

$$\boldsymbol{a}_k = \frac{1}{m}\sum \boldsymbol{f}(t_k, \boldsymbol{x}_k)$$

then

$$\boldsymbol{x}_{k+1} = 2\boldsymbol{x}_k - \boldsymbol{x}_{k-1} + \boldsymbol{a}_k h^2$$

or

$$\bar{\boldsymbol{v}}_k = \frac{\boldsymbol{x}_k - \boldsymbol{x}_{k-1}}{h}$$
$$\boldsymbol{x} = \boldsymbol{x}_k + \bar{\boldsymbol{v}}h + \boldsymbol{a}_k h^2$$

2.4 Runge-Kutta

Using higher order terms in the Taylor series quickly becomes infeasible. Instead we will try to find a better direction. Runge-Kutta methods do this by evaluating $f(t, \boldsymbol{x}(t))$ at several tangents between t and $t + h$, then computes a weighted average of these to form a better estimate for the direction. The general form is

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + h\Phi(t_k, \boldsymbol{x}_k, h)$$

where $\Phi(t_k, \boldsymbol{x}_k, h)$ is a weighted average of evaluations of $f(t, \boldsymbol{x})$ between t_k and t_{k+1}. A Runge-Kutta method of order p is as accurate as a pth order Taylor series while only requiring the first derivative. Increasing the order of a Runge-Kutta method, implies evaluating more points between t_k and t_{k+1} and therefore higher accuracy at the cost of more computations. If we want to average s number of tangents, then we evaluate $f(t, \boldsymbol{x})$ from $m = 1, 2, ..., s$ (called states) with

$$\tilde{t}_m = t_k + a_m h, \qquad \tilde{\boldsymbol{x}}_m = \boldsymbol{x}_k + h\sum_{n=1}^{m-1} b_{mn} f(\tilde{t}_n, \tilde{\boldsymbol{x}}_n).$$

Which are our intermediary points between t_k and t_{k+1}. Then we use these points to compute the s tangents $(f(\tilde{t}_m, \tilde{\boldsymbol{x}}_m))$ which are then weighted and averaged to finally update our state

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + h\sum_{m=1}^{s} c_m f(\tilde{t}_m, \tilde{\boldsymbol{x}}_m)$$

The coefficients a_m, b_{mn} and c_m are defined by matrices specific to the Rune-Kutta method being used

$$a = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_s \end{bmatrix} \quad b = \begin{bmatrix} b_{11} & & & & \\ b_{21} & b_{22} & & & \\ b_{31} & b_{32} & b_{33} & & \\ \vdots & \vdots & \vdots & \ddots & \\ b_{s,1} & b_{s,2} & b_{s,3} & \dots & b_{s,s-1} \end{bmatrix} \quad c = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_s \end{bmatrix}$$

In this course we are most interested in RK4 which is widely used in all sorts of applications since it gives a good accuracy for a modest computational cost. It has four stages and the coefficient matrices are given by

$$a = \begin{bmatrix} 0 \\ \frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 & 0 & 0 \\ \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad c = \begin{bmatrix} \frac{1}{6} \\ \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{6} \end{bmatrix}$$

Inserting these coefficients into the general algorithm and expanding the four stages gives us the points

$$\begin{aligned}
\tilde{t}_1 &= t_k, & \tilde{\boldsymbol{x}}_1 &= \boldsymbol{x}_k \\
\tilde{t}_2 &= t_k + \frac{1}{2}h, & \tilde{\boldsymbol{x}}_2 &= \boldsymbol{x}_k + \frac{1}{2}h\boldsymbol{f}(\tilde{t}_1, \tilde{\boldsymbol{x}}_1) \\
\tilde{t}_3 &= t_k + \frac{1}{2}h, & \tilde{\boldsymbol{x}}_3 &= \boldsymbol{x}_k + \frac{1}{2}h\boldsymbol{f}(\tilde{t}_2, \tilde{\boldsymbol{x}}_2) \\
\tilde{t}_4 &= t_k + h & \tilde{\boldsymbol{x}}_4 &= \boldsymbol{x}_k + h\boldsymbol{f}(\tilde{t}_3, \tilde{\boldsymbol{x}}_3)
\end{aligned}$$

and then using the points to compute the tangents

$$\tilde{\boldsymbol{f}}_1 = \boldsymbol{f}(\tilde{t}_1, \tilde{\boldsymbol{x}}_1), \qquad \tilde{\boldsymbol{f}}_2 = \boldsymbol{f}(\tilde{t}_2, \tilde{\boldsymbol{x}}_2), \qquad \tilde{\boldsymbol{f}}_3 = \boldsymbol{f}(\tilde{t}_3, \tilde{\boldsymbol{x}}_3), \qquad \tilde{\boldsymbol{f}}_4 = \boldsymbol{f}(\tilde{t}_4, \tilde{\boldsymbol{x}}_4)$$

we can then average them and compute $\boldsymbol{x}_{k+1}$ at the same time

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \frac{h}{6}\left(\tilde{\boldsymbol{f}}_1 + 2\tilde{\boldsymbol{f}}_2 + 2\tilde{\boldsymbol{f}}_3 + \tilde{\boldsymbol{f}}_4\right)$$

2.5 Two-Body Problem

Figure shows two bodies with masses m_1 and m_2 and positions $\boldsymbol{r}_2^i$, $\boldsymbol{r}_1^i$ relative to an inertial frame with origin at the system's center of mass such that

$$\frac{m_1 \cdot \boldsymbol{r}_1^i + m_2 \cdot \boldsymbol{r}_2^i}{m_1 + m_2} = \boldsymbol{0} \tag{2}$$

When the only acting force between them is the gravitational force the motion of the second body relative to the first can be represented by the fundamental equation for relative two-body motion

$$\ddot{\boldsymbol{r}}^i = -\frac{\mu}{r^3}\boldsymbol{r}^i$$

with $\boldsymbol{r}^i = \boldsymbol{r}_2^i - \boldsymbol{r}_1^i$ the being relative position and $\mu = G(m_1 + m_2)$ is the gravitational parameter with $G = 6.6742 \cdot 10^{-11}\mathrm{m}^3/(\mathrm{kg}{\cdot}\mathrm{s}^2)$ being the universal Gravitational constant.

Much of orbital mechanics is based on the analytical solution of this equation, which result in the orbit equation and some constants (called orbital parameters). Simulating this system can therefore be reduced to evaluating the orbit equation over time for a particular set of orbital parameters and initial conditions. However, implementing these equations can be error prone and additionally it can sometimes be difficult to know if a resulting implementation gives correct results or not. Therefore simulating a satellite's motion directly from the differential equations can be useful in validating software implementations of orbital mechanics. The differential equation is of second order, but can be written as two first order differential equations in the form $\dot{\boldsymbol{x}} = f(t, \boldsymbol{x})$ as

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{r}^i \\ \dot{\boldsymbol{r}}^i \end{bmatrix}, \qquad \dot{\boldsymbol{x}} = \begin{bmatrix} \dot{\boldsymbol{r}}^i \\ -\frac{\mu}{r^3}\boldsymbol{r}^i \end{bmatrix} = f(\boldsymbol{x})$$

Note: The system does not explicitly depend on time and is therefore autonomous, while the solvers from Section 2 assume a non-autonomous system. This doesn't really have any consequence when implementing these numeric solvers since they work fine for autonomous systems without modification. We will expand a bit more on this in the next section.

3 Simulating a Satellite Orbiting Earth

3.1 Earth Model

In the simulations we will assume the following about the Earth

- Spherical with radius $R_E = 6378.1363$ km
- Constant rotational velocity $\omega_E = 7.292115 \cdot 10^{-5}$ rad/s
- Uniform density $m_E = 5.974 \cdot 10^{24}$ kg

3.2 Satellite Model

In the two-body problem we can make some simplifications if we let the first body be Earth and the second body be a satellite. Since in that case $m_1 >> m_2$ then $\mu \approx Gm_1$ and the system's center of mass is approximately equal to Earth's center of mass. Therefore we can assume the inertial frame is equal to the ECI frame such that $\boldsymbol{r}_1^i = \mathbf{0}$, and $\boldsymbol{r}^i = \boldsymbol{r}_2^i$.

Summarized the differential equation can be written as

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{r}^i \\ \dot{\boldsymbol{r}}^i \end{bmatrix}, \qquad \dot{\boldsymbol{x}} = \begin{bmatrix} \dot{\boldsymbol{r}}^i \\ -\frac{\mu}{r^3}\boldsymbol{r}^i \end{bmatrix} = f(\boldsymbol{x})$$

with $\mu \approx \frac{Gm_E}{10^6}$ km^3/s^2, and $\boldsymbol{r}^i, \dot{\boldsymbol{r}}^i$ being the position and velocity of the satellite in the ECI-frame with km and km/s as units. In elliptical orbits the total energy of a satellite should be constant

$$\mathcal{E} = m\left(\frac{\|\dot{\boldsymbol{r}}^i\|^2}{2} - \frac{\mu}{\|\boldsymbol{r}^i\|}\right)$$

and it can therefore be used to compare the accuracy between numeric solvers and to highlight any drift in the numerical solution.

Assignment 3.1

1. Extend simutils.py with numeric solver functions to simulate dynamical systems:

   step_euler(h,t_k,x_k,f)

   step_leapfrog(h,t_k,x_k,f)

   step_verlet(h,t_k,x_k,x_k-1,f)

2. Simulate a Satellite

$$\boldsymbol{r}^i(0) = \begin{bmatrix} R_E + 800 \\ 0 \\ 0 \end{bmatrix} \text{ km}, \qquad \dot{\boldsymbol{r}}^i(0) = \begin{bmatrix} 0 \\ 0 \\ \sqrt{\frac{\mu}{\|\boldsymbol{r}^i(0)\|}} \end{bmatrix} \text{ km/s}$$

   with Earth parameters from Section 3.1. Try each of the implemented solvers to verify that they give similar results (they might require different t_step). Use the simulation parameters: t_0= 0, t_e= 20000, t_step= 10, speed_factor = 100, anim_dt = 0.04, scale_factor= 1000 as a starting point.

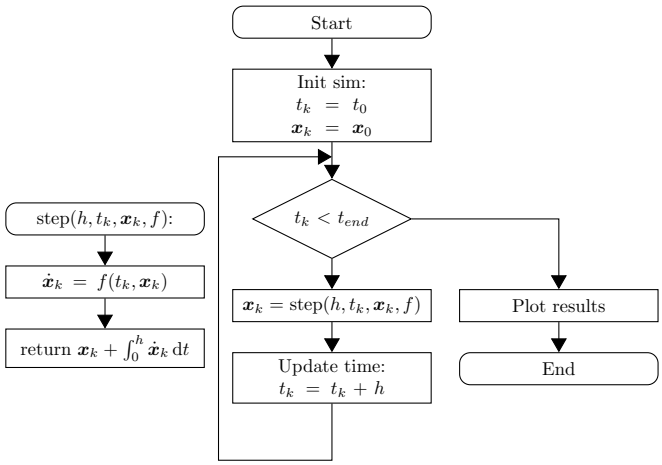

Figure 1: General structure of a numeric solver for ODEs.

Note:

- The function arguments are as follows

  step size: h

  time: t_k,

  state: x_k,

  previous state: x_k-1,

  function: f (implements $f(t_k, \boldsymbol{x}_k)$ and returns dot_x_k given t_k, x_k)

- The functions return:

  state: x_k+1

To keep the functions useful to simulate arbitrary dynamical systems we only implement the integration logic for one timestep, i.e. it will compute the next state and return that to the user. Then it up to the user to call the function again with the updated state for each time step. The general idea of what it looks like is shown in Figure 1. The state is just a numpy array. The system dynamics are provided as an argument to the function, a basic example of having functions for arguments in python is

```
def mysin(x):
  return x-x**3/6+x**5/120
def mycos(x):
  return 1-1/2*x**2+1/24*x**4
def eval(a,f):
  return f(a)
x = 2.5
y = eval(x,mysin) #compute sin(x)
print(eval(y,mycos)) #compute cos(y) => cos(sin(x))
```

In the case when a differential equation does not depend on t or $\boldsymbol{x}$ then this is handled inside the function f, such that it is transparent to the solver for instance

```
def f(t,x):
  return -x
```

for the system $\dot{x} = -x$.

3.3 Nonimpulsive orbital maneuvers

Although we can use the analytical solution to the two-body problem to study orbital mechanics there are some cases when we have to revert back to the differential equation. An example of this is when we are considering orbital maneuvers that are nonimpulsive (they occur over a long period of time). In those cases the fundamental equation for relative two-body motion takes the alternative form

$$\ddot{\boldsymbol{r}}^i = -\frac{\mu}{r^3}\boldsymbol{r}^i + \frac{1}{m}\boldsymbol{f}_T$$

where m is the mass of the spacecraft and $\boldsymbol{f}_T$ is the thrust force generated by onboard rocket motors. When the equation includes the thrust force it does not have an analytical solution anymore and we have to rely on numerical techniques.

As in the previous section we will simulate a satellite orbiting Earth. The parameters for the satellite is given in Table 1. We will assume that the Satellite has an attitude controller that will align the thrust vector with $\dot{\boldsymbol{r}}^i$ such that

$$\boldsymbol{f}_T = T\frac{\dot{\boldsymbol{r}}^i}{\|\dot{\boldsymbol{r}}^i\|}$$

then the sign of T determines if the maneuver is an accelerating or braking maneuver. The goal of the orbital maneuver is to transfer the satellite from its initial elliptical MEO orbit to a LEO orbit with altitude $z_c = 1500$ km. The thrust controller is defined in the following way:

$$T = \begin{cases} k_1 \cdot (r_c - r_a) & \text{if } 0.9 < \cos\theta \\ k_2 \cdot (r_c - r_p) & \text{if } \cos\theta < -0.9 \\ 0 & \text{otherwise} \end{cases}$$

where $r_c = R_E + z_c$, r_a are r_p is the apoapsis and periapsis distances for the current orbit. As the maneuver is executed r_a and r_p will be changing and needs to be updated

$$r_a = \frac{\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|^2}{\mu\,(1 - \|\boldsymbol{e}^i\|)}, \qquad r_p = \frac{\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|^2}{\mu\,(1 + \|\boldsymbol{e}^i\|)}$$

with

$$\boldsymbol{e}^i = \frac{1}{\mu}\dot{\boldsymbol{r}} \times (\boldsymbol{r} \times \dot{\boldsymbol{r}}) - \frac{1}{\|\boldsymbol{r}^i\|}\boldsymbol{r}^i \tag{3}$$

And $\cos\theta$ can be found from

$$\cos\theta = \frac{\boldsymbol{e}^i \bullet \boldsymbol{r}^i}{\|\boldsymbol{e}^i\| \cdot \|\boldsymbol{r}^i\|} \tag{4}$$

Table 1: Model parameters for a spacecraft.

Name	Symbol	Value
mass	m	8000kg
Initial position	$\boldsymbol{r}^i(0)$	$\begin{bmatrix}7378 & 0 & 0\end{bmatrix}^\top$
Initial velocity	$\dot{\boldsymbol{r}}^i(0)$	$\begin{bmatrix}0 & 0 & 9\end{bmatrix}^\top$

k_1, k_2 are gain parameters to be tuned (they should typically be small, usually in the range $[10^{-6}, 10^{-3}]$).

Assignment 3.2

1. Extend simutils.py with numeric solver functions to simulate dynamical systems:

   step_RK4(h,t_k,x_k,f)

2. Simulate the Satellite scenario outlined Section 3.3 using RK4, with Earth parameters from Section 3.1. Use the simulation parameters: t_0= 0, t_e= 53000, t_step= 100, speed_factor = 1, anim_dt = 1/25, scale_factor= 1000. Try to achieve the desired orbit before t_end, with as little control effort as possible.